ResumeLens: Formal Language-Based Résumé Screening

Regular expressions, finite-state transducers, finite automata and a context-free grammar for checking qualification patterns

Juan Andrés Cano Torres · Samuel Cardoso Martinez · Juan Felipe Martinez
Computación y Estructuras Discretas III — Group 3
Universidad Icesi · Cali, Colombia · 2026-2

Central question: can a chain of formal language models decide — transparently and reproducibly — whether the qualifications written in a résumé satisfy the pattern of a professional profile, regardless of how the candidate spelled or ordered them?

Introduction

Recruiters screen many résumés, but the same qualification is written in many ways: `JS`, `Javascript` and `JavaScript` are one language; `sklearn` and `Scikit-learn` are one library.

ResumeLens checks whether the **explicitly written** qualifications satisfy a formal pattern for four profiles: Full Stack Developer, Machine Learning Engineer, and the team-defined DevOps Engineer and Data Engineer. It does **not** rank candidates or make hiring decisions.

Methodology

▼ plain-text résumé

1 Extraction
Regular expressions · Python `re`

▼ raw strings: "Scikit-learn", "NumPy"…

2 Normalization + sorting
Finite-state transducer · pyformlang

▼ canonical tokens in profile order

3 Pattern recognition
ε-NFA → minimal DFA per profile · pyformlang

▼ ACCEPTED / REJECTED × 4 profiles

4 Candidate profile language
Context-free grammar (EBNF) · textX

▼ validated profile → HTML page

Figure 1. One general pipeline for all four profiles. Profiles are **data** (ordered categories of tokens), so no stage contains profile-specific code.

Running example

Mary Jane Watson
2 years of experience developing predictive models and data-processing pipelines.
Technical Skills:
Python, Pandas, NumPy, Scikit-learn, TensorFlow, SQL, Git.

name · experience · skill (6 regex families)

Extraction Stage 1 · regex

One regular expression per kind of information: name, e-mail, phone, links, education, experience and six skill families.

```
(?<![\w.])(?:pandas|numpy|scikit[ -]?learn|sklearn|
tensor[ -]?flow|py[ -]?torch|keras)(?!\w)
```

Figure 2. Library family (case-insensitive). The look-around boundaries stop `js` inside `NodeJS` or `SQL` inside `PostgreSQL` from being reported as separate skills.

Normalization Stage 2 · FST

A finite-state transducer **M = (Q, Σ, Γ, δ, ω, q_0, F)** reads a case-folded string symbol by symbol, writes ε until the last symbol, and then writes the canonical token.

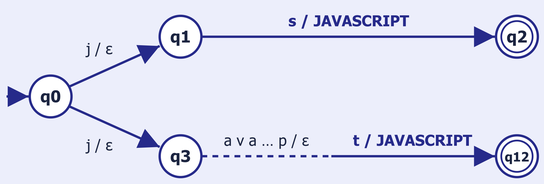


Figure 3. Two paths of the transducer: `js` and `javascript` both translate to **JAVASCRIPT**. Label *a / b*: read *a*, write *b*; double circle = accepting state.

Raw string (stage 1)	Canonical token
`Scikit-learn`, `sklearn`	SCIKIT_LEARN
`Postgres`, `PostgreSQL`	POSTGRESQL
`React.js`, `ReactJS`	REACT
`predictive models`	ML_MODEL_DEVELOPMENT

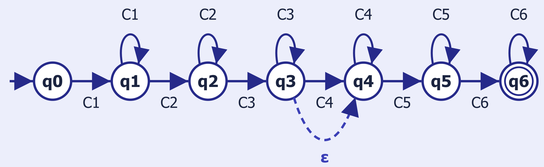
106 surface variants · **48** canonical tokens · **893** FST states

Tokens are then deduplicated and **sorted in the profile's category order**, so the verdict does not depend on how the candidate ordered the skills.

Recognition Stage 3 · automata

A profile is an ordered list of categories $C_1 \dots C_n$. Required categories must appear one or more times; optional ones may be skipped:

$$L(P) = X_1\, X_2 \dots X_n, \quad X_i = C_i^{+} \text{ (required) or } C_i^{*} \text{ (optional)}$$



C1 PYTHON — **C4** ML_MODEL_DEVELOPMENT *(optional)*
C2 PANDAS, NUMPY — **C5** SQL, POSTGRESQL, MYSQL
C3 SCIKIT_LEARN, TENSORFLOW, PYTORCH, KERAS — **C6** GIT

Figure 4. ε-NFA of Machine Learning Engineer. A self-loop accepts more tokens of the same category; the dashed ε-transition skips the optional category C4.

```
Mary Jane: q0→q1→q2→q2→q3→q3→q4→q5→q6 ✔
PYTHON PANDAS TENSORFLOW POSTGRESQL GIT:
q0→q1→q2→q3 —ε→ q4→q5→q6 ✔ ACCEPTED
```

Why an ε-NFA? Categories are disjoint, so the only non-determinism is ε. pyformlang converts it to an equivalent **minimal DFA**.

Profile	|Q|	|Σ|	ε
Full Stack Developer	7	19	1
Machine Learning Engineer	7	12	1
DevOps Engineer	9	14	2
Data Engineer	7	18	1

Profile language Stage 4 · CFG

```
candidate
  "Mary Jane Watson" {
  skills: PYTHON, PANDAS, …
  classification {
    MACHINE_LEARNING_ENGINEER:
      ACCEPTED
    DATA_ENGINEER: REJECTED
  }
}
```

```
Candidate =
  "candidate" STRING "{"
  Contact {Education}
  {Experience} Skills
  Classification "}" ;
Skills = "skills" ":"
  TOKEN {"," TOKEN} ;
TOKEN = [A-Z] {[A-Z0-9_]} ;
```

```
skills: javascript → line 8, column 11: Expected TOKEN
```

Figure 5. Generated profile and part of the EBNF grammar (textX). Invalid text is rejected with its position; valid profiles are rendered as HTML.

Results

Sample résumé	Full Stack	ML Eng.	DevOps	Data Eng.
Wednesday Addams	✔	✘	✘	✘
Mary Jane Watson	✘	✔	✘	✘
Alex Morgan	✘	✘	✔	✘
Taylor Reed	✘	✘	✘	✔
Sam Rivera *(near miss)*	✘	✘	✘	✘
Jordan Lee *(two profiles)*	✘	✘	✔	✔

Table 1. Verdict of each profile automaton (✔ accepted). Each reference sample is accepted only by its own profile.

- **A rejection always means a missing required category.** Sam Rivera has the whole frontend but no backend or database.
- **Profiles are independent.** Jordan Lee satisfies DevOps and Data Engineer at the same time.

164 automated tests (pytest) · **4** profiles, one generic pipeline · **6** sample résumés, incl. edge cases

Conclusions

- Each stage uses the weakest model that solves it: regex to find, FST to rewrite, automata to recognize, CFG to structure.
- Every verdict is **explainable**: the accepted path or the missing category.
- Profiles as data: one automaton construction serves all four.

Limitations & future work

- Unknown spellings (e.g. `Rust`) are ignored; only explicit skills count.
- Next: a prefix-sharing transducer (fewer than 893 states) and ESCO / O*NET skill vocabularies.

References

1. Hopcroft, Motwani & Ullman. *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Pearson, 2006.
2. Mohri. Finite-state transducers in language and speech processing. *Computational Linguistics* 23(2), 1997.
3. Dejanović et al. TextX: a Python tool for DSL implementation. *Knowledge-Based Systems* 115, 2017.
4. Mernik, Heering & Sloane. When and how to develop domain-specific languages. *ACM Computing Surveys* 37(4), 2005.